TypeScript

Типы для JavaScript и React

```
type Spell = {
  id: string;
  name: string;
  cost: number;
};
```

Frontend-разработчик



Что будем уметь

- Объяснять, что проверяет TypeScript и чего он не проверяет
- Подключать TypeScript к проекту React на Vite
- Описывать типы данных, функций и props
- Работать с union-типами, null и сужением типа
- Описывать действия reducer через discriminated union
- Проверять данные извне, а не верить их типу

Демонстрация после лекции: «Жёлтый маг» на TypeScript

Где мы сейчас

Тема	Что уже знаем	Что добавим
04 JavaScript	динамическая типизация, typeof	проверку типов до запуска
05 Интерфейс	reducer, «загрузке не доверяем»	типы действий и проверку данных
06 Сборка	ESLint и Prettier работают с AST	третью проверку — типы
07 React	компоненты, props, useState	типы props, событий и состояния

Сегодня — третья проверка из темы 06: типы

Три ошибки, которые видны только при запуске

```
// 1. опечатка в props — на экране пусто
<StatBar label="HP" valeu={24} max={30} />

// 2. find может ничего не найти
const spell = spells.find((s) => s.id === id);
setMana(mana - spell.cost); // TypeError

// 3. сервер прислал "5" вместо 5
mana + potion.amount; // "205", а не 25
```

Это код из прошлой лекции.

Ошибка проявится, только если нажать нужную кнопку.

TypeScript находит такие ошибки в редакторе, до запуска

Что такое TypeScript

- Надстройка над JavaScript: тот же язык плюс аннотации типов
- Компилятор tsc проверяет типы и ничего не запускает
- Перед запуском типы удаляются — в браузер идёт обычный JavaScript
- Редактор показывает ошибки и подсказки прямо при наборе

TypeScript — инструмент разработки, а не другой язык в браузере

Типы стираются

```
// исходник
const mana: number = 20;

function cast(cost: number): number {
  return mana - cost;
}

// что получает браузер
const mana = 20;

function cast(cost) {
  return mana - cost;
}
```

Аннотации нужны только для проверки.

Во время работы типов нет: typeof работает как в теме 04.

Проверка типов — до запуска, во время работы их нет

Проект React на TypeScript

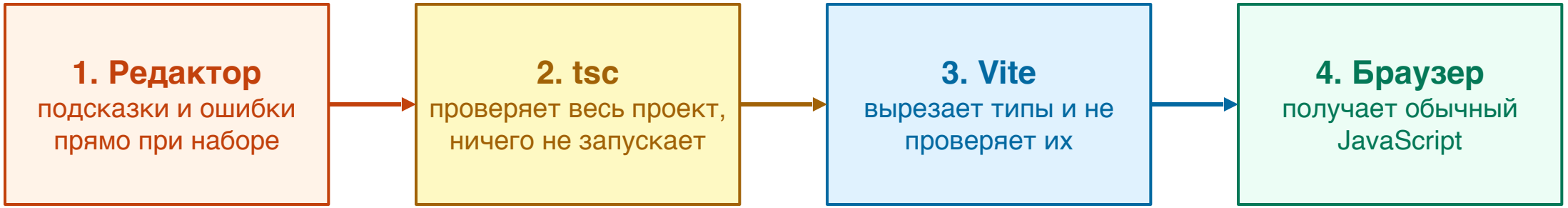
```
$ npm create vite@latest yellow-mage -- --template react-ts
$ cd yellow-mage
$ npm install
$ npm run dev
```

Файлы .tsx вместо .jsx.

Настройки проверки — в tsconfig.app.json.

Новый проект — шаблон react-ts; старый код переносим в его src

Где работает TypeScript



npm run build в шаблоне react-ts = tsc -b, затем vite build

Ошибку типа видят редактор и tsc, но не Vite и не браузер

Строгий режим

```
// tsconfig.app.json в шаблоне react-ts (часть)
{
  "compilerOptions": {
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true
  }
}
```

strict включает главные проверки, в том числе null и undefined.

В шаблоне он уже включён — не выключаем.

Работаем только в strict: без него TypeScript пропускает много ошибок

Аннотации и вывод типов

```
let mana = 20;       // тип number выведен сам
mana = "много";
// Type 'string' is not assignable to type 'number'.

function cast(cost: number) {
  return mana - cost; // результат — number
}

cast("5");
// Argument of type 'string' is not assignable
// to parameter of type 'number'.
```

TS выводит тип из значения.

Аннотация нужна там, где выводить не из чего: параметры функций.

Типы пишем на границах: параметры, props, данные извне

Базовые типы

Тип	Пример значения
string	"Искра"
number	20, 0.5
boolean	true, false
string[]	["Искра", "Щит"]
{ name: string; cost: number }	объект с полями
null, undefined	значения нет

Те же типы значений, что typeof из темы 04, плюс массивы и объекты

Тип для объекта

```
type Spell = {
  id: string;
  name: string;
  cost: number;
  description?: string; // необязательное
};

const spark: Spell = { id: "spark", name: "Искра", cost: 5 };

const bad: Spell = { id: "x", name: "Щит" };
// Property 'cost' is missing in type ... but
// required in type 'Spell'.
```

type даёт имя форме объекта.

interface делает то же — в курсе пишем type.

Один тип Spell — и все функции знают форму заклинания

Функции

```
function totalCost(spells: Spell[]): number {
  return spells.reduce((sum, s) => sum + s.cost, 0);
}

function canCast(spell: Spell, mana: number) {
  return spell.cost <= mana; // результат — boolean
}

spells.filter((s) => s.cost <= 5);
// s: Spell — тип выведен из массива
```

Параметры аннотируем всегда.

Тип результата и параметры колбэков TS обычно выводит сам.

Типы параметров — контракт функции

any и unknown

```
const data: any = JSON.parse(text);
data.spels.length; // TS молчит, TypeError при запуске

const safe: unknown = JSON.parse(text);
safe.spells.length;
// 'safe' is of type 'unknown'.
```

any — «не проверяй».

unknown — «сначала проверь, потом используй».

any не пишем; для неизвестных данных — unknown

Union: одно из нескольких

```
type Kind = "attack" | "defense" | "heal";

let kind: Kind = "attack";
kind = "heal";
kind = "fire";
// Type '"fire"' is not assignable to type 'Kind'.

let id: string | number = "spark";
id = 7;
```

| читается как «или».

Вместо «любая строка» — только допустимые значения.

Union из строк вместо enum: ловит опечатки и просто стирается

null и undefined

```
const spell = spells.find((s) => s.id === id);
//    spell: Spell | undefined

setMana(mana - spell.cost);
// 'spell' is possibly 'undefined'.
```

find может ничего не найти — это часть типа.

Ошибка № 2 найдена до запуска.

strict заставляет обработать случай «значения нет»

Сужение типа

```
const spell = spells.find((s) => s.id === id);
if (spell === undefined) return;
setMana(mana - spell.cost); // здесь spell: Spell

function label(id: string | number) {
  if (typeof id === "number") return `№${id}`;
  return id.toUpperCase(); // здесь id: string
}
```

После проверки TS сужает тип в этой ветке.

Работают ===, typeof, in, Array.isArray и ранний return.

Сужение — обычные проверки JavaScript, которые понимает TypeScript

Действия reducer из темы 05

```
function reduce(state, act) {
  switch (act.type) {
    case "cast":
      return { ...state, mana: state.mana - act.cost };
    case "rest":
      return { ...state, mana: state.maxMana };
  }
}

reduce(s, { type: "cats", cost: 5 }); // undefined
reduce(s, { type: "cast" });          // mana: NaN
```

Опечатка и забытое поле не дают ошибки.

Плохое состояние обнаружится позже и в другом месте.

У каждого действия своя форма данных

Discriminated union

```
type Action =
  | { type: "cast"; cost: number }
  | { type: "rest" }
  | { type: "damage"; amount: number };

function reduce(state: State, act: Action): State {
  switch (act.type) {
    case "cast":
      return { ...state, mana: state.mana - act.cost };
    case "rest":
      return { ...state, mana: state.maxMana };
    case "damage":
      return { ...state, hp: state.hp - act.amount };
  }
}
```

Поле type отличает варианты.

Внутри case "cast" TS знает, что у act есть cost.

Поле type + union = каждое действие проверено

Что теперь ловит TypeScript

```
reduce(s, { type: "cats", cost: 5 });
// Type '"cats"' is not assignable to type ...

reduce(s, { type: "cast" });
// Property 'cost' is missing ...

// добавили в Action { type: "heal" }, но забыли case:
// Function lacks ending return statement and
// return type does not include 'undefined'.
```

Опечатка, забытое поле и забытый вариант — всё до запуска.

Новое действие — TS покажет все места, где его забыли обработать

Типы props

```
type StatBarProps = {
  label: string;
  value: number;
  max: number;
};

function StatBar({ label, value, max }: StatBarProps) {
  return (
    <div className="stat">
      {label}: {value} / {max}
      <progress value={value} max={max} />
    </div>
  );
}
```

Тип props — контракт компонента.

Редактор подсказывает пропсы, как только набираем <StatBar.

Тип props описывает, что компонент принимает

Ошибки при использовании компонента

```
<StatBar label="HP" valeu={24} max={30} />
// Property 'valeu' does not exist on type
// 'IntrinsicAttributes & StatBarProps'.
// Did you mean 'value'?

<StatBar label="HP" value="24" max={30} />
// Type 'string' is not assignable to type 'number'.

<StatBar label="HP" max={30} />
// Property 'value' is missing in type ...
```

Ошибка № 1 найдена до запуска.

IntrinsicAttributes — служебные пропсы React вроде key.

Лишний, пропущенный или неверный проп — ошибка в редакторе

Колбэки и children

```
import type { ReactNode } from "react";

type SpellItemProps = {
  spell: Spell;
  canCast: boolean;
  onCast: (id: string) => void;
};

type PanelProps = {
  title: string;
  children: ReactNode;
};
```

Функция в props описывается сигнатурой: параметры и результат.

ReactNode — всё, что можно вложить в JSX.

Пропсы-события — это типы функций

useState

```
const [query, setQuery] = useState("");
// string — выведен из ""

type Status = "idle" | "loading" | "error";
const [status, setStatus] = useState<Status>("idle");

const [selected, setSelected] =
  useState<Spell | null>(null);
```

Обычно тип выводится из начального значения.

Union или null в начале — указываем тип явно.

useState<T> — когда из начального значения тип не понять

Типы событий

```
import type { ChangeEvent, SubmitEvent } from "react";

<input onChange={(e) => setQuery(e.target.value)} />
// e выведен сам: ChangeEvent<HTMLInputElement>

function handleChange(e: ChangeEvent<HTMLInputElement>) {
  setName(e.target.value);
}

function handleSubmit(e: SubmitEvent<HTMLFormElement>) {
  e.preventDefault();
}
```

Отдельной функции тип события указываем сами.

Подсказка: навести курсор на onChange в редакторе.

В обработчике прямо в JSX тип события выводится сам

Мини-задача: что скажет TypeScript

```
type Spell = { id: string; name: string; cost: number };

const res = await fetch("/api/spells");
const spells: Spell[] = await res.json();

console.log(spells[0].cost.toFixed(1));

// сервер прислал:
// [{ "id": "spark", "name": "Искра", "cost": "5" }]
```

Будет ли ошибка в редакторе?

А при запуске?

Подумать 30 секунд, ответ на следующем слайде

Разбор: TypeScript промолчит

```
const spells: Spell[] = await res.json();
// res.json() возвращает any — TS верит на слово

spells[0].cost.toFixed(1);
// при запуске:
// TypeError: spells[0].cost.toFixed is not a function
```

Тип — обещание, а не проверка.

Ошибка № 3 из начала лекции так и осталась.

Тип данных извне — то, что мы ждём, а не то, что пришло

Проверка при загрузке

```
function isSpell(x: unknown): x is Spell {
  return (
    typeof x === "object" && x !== null &&
    "id" in x && typeof x.id === "string" &&
    "name" in x && typeof x.name === "string" &&
    "cost" in x && typeof x.cost === "number"
  );
}

const data: unknown = await res.json();
if (!Array.isArray(data) || !data.every(isSpell)) {
  throw new Error("Неверный ответ сервера");
}
// дальше data: Spell[]
```

unknown заставляет проверить.

x is Spell — функция-проверка, которая сужает тип.

На границе — проверка, внутри программы — надёжные типы

Что TypeScript проверяет, а что нет

Ошибка	TypeScript
Опечатка в поле, пропе или действии	найдёт
Аргумент не того типа	найдёт
Забытая проверка на null и undefined	найдёт в strict
Данные из fetch, localStorage, JSON.parse	не найдёт — проверяем сами
Неверная логика: формула урона	не найдёт — нужны тесты

Типы ловят несовпадение формы данных, а не ошибки смысла

Что сегодня не трогаем

Тема	Когда
Обобщённые типы глубже useState<T>	по мере надобности
Служебные типы: Partial, Pick, Omit	по мере надобности
Файлы .d.ts и типы чужих библиотек	по мере надобности
useReducer с типом Action	React II
Библиотеки схем, например Zod	по желанию, с загрузкой данных

Сегодня — основа, которой хватит на итоговый проект

Демонстрация после лекции

- npm create vite@latest yellow-mage -- --template react-ts
- Перенос «Жёлтого мага»: .jsx → .tsx, тип Spell и типы props
- npm run build: tsc находит ошибки, которые пропустил JavaScript
- Подсказки редактора: наведение, автодополнение, переименование

Код демонстрации прилагается: папка src на TypeScript

Практика: «Мои задачи» на TypeScript

- Создать проект react-ts и перенести src из прошлой практики
- Тип Task и типы props для каждого компонента
- Фильтр — union "all" | "active" | "done"
- Ни одного any и ни одного @ts-ignore
- npm run build проходит без ошибок типов

Сдача — ссылкой на Pull Request, как обычно

Что важно унести с собой

- TypeScript проверяет типы до запуска, перед запуском они стираются
- Типы пишем на границах: параметры, props, данные извне
- strict обязателен: null и undefined становятся частью типа
- Union и сужение — обычные проверки JavaScript, понятные TS
- Действия reducer — discriminated union по полю type
- Тип — обещание: данные извне проверяем при загрузке

Следующая лекция: React II — эффекты и загрузка данных

Источники и дальнейшее чтение

- TypeScript Handbook: The Basics, Everyday Types, Object Types, Functions
- TypeScript Handbook: Narrowing — discriminated unions, type predicates
- TSConfig Reference: strict, erasableSyntaxOnly
- react.dev: Using TypeScript
- Vite: Features — TypeScript

Ссылки на конкретные разделы находятся в заметках слайдов